

Preservación semántica en migraciones de passkeys con CXF y CXP

PasskeyTransit v0.3 Informe reproducible de controles e interoperabilidad

Resumen

Estudiamos si una passkey que continúa autenticando después de un intercambio conserva además su identidad, extensiones y garantías operacionales. Presentamos PasskeyTransit v0.3, un harness reproducible para CXF, CXP/HPKE, WebAuthn, mutaciones y fallos transaccionales. La campaña principal ejecutó 6,144 intentos sobre 256 credenciales, 12 rutas y dos repeticiones usando políticas de control y autenticadores virtuales Chromium. El diseño contiene 3,072 celdas credencial×ruta, ejecutadas dos veces y resumidas en 96 grupos ruta×estrato. Ninguna de estas unidades fue muestreada de una población real. Se importaron 5,120 intentos y se rechazaron 1,024 antes de la ceremonia. Las 5,120 aserciones WebAuthn ejecutadas fueron aceptadas; 2,048 intentos (40.00%; intervalo descriptivo de sensibilidad por bootstrap de credencial 35.71%-44.51%) combinaron login correcto con el fallo de otra propiedad aplicable. La preservación semántica completa observable fue 37.50% (2,304/6,144; intervalo descriptivo de sensibilidad 32.62%-42.58%). Estos porcentajes caracterizan estímulos sintéticos diseñados, no productos ni prevalencia real. PRF y preservación positiva de credBlob permanecen no evaluables por límites de la interfaz CDP.

Introducción

CXF estandariza la representación de credenciales [1] y CXP propone su transporte protegido. Sin embargo, aceptar un documento e incluso completar una aserción WebAuthn no demuestra por sí solo que sobrevivan todas las propiedades de una passkey. Modelamos la credencial como $C=(I,K,F,S)$: identidad, material criptográfico, funciones observables y garantías operacionales.

La proposición falsable es que una importación aceptada y un login correcto son insuficientes para establecer preservación semántica completa. No formulamos afirmaciones de novedad ni sobre implementaciones comerciales no examinadas.

Método

El protocolo passkeytransit-semantic-preservation-v1.1 fue congelado antes de esta campaña confirmatoria corregida. El corpus contiene 256 credenciales ES256, 32 en cada uno de ocho estratos: básica, PRF con UV, PRF sin UV, largeBlob, credBlob, marcador de pagos, combinación de extensiones y miembro opcional futuro. Doce rutas cubren migración directa, round trip y multihop.

Cada salto serializa un documento CXF validado y lo transporta mediante HPKE base X25519/HKDF-SHA256/AES-128-GCM [3]. El navegador importa el resultado en un autenticador virtual y ejecuta `navigator.credentials.get()`. Un verificador Python independiente de la ceremonia comprueba desafío, origen, RP-ID hash, flags UP/UV, firma ES256 y contador cero [4].

Los oráculos devuelven PASS, FAIL, NOT_APPLICABLE o NOT_EVALUABLE. Separadamente clasificamos estado de ejecución, preservación semántica y evaluación normativa. Los intervalos de bootstrap agrupado por credencial son análisis descriptivos de sensibilidad del diseño y no intervalos de confianza poblacionales; las comparaciones de rutas son pareadas por credencial y repetición.

Modelo de amenazas

El experimento protege la confidencialidad e integridad del payload frente a un observador o modificador del canal que no posee la clave privada del importador. El importador y los perfiles de control son componentes confiables e instrumentados; no modelamos compromiso del endpoint, extracción de claves, canales laterales ni engaño del usuario. El binding experimental rechaza mezcla de solicitudes, modificación del contexto, downgrade y replay dentro del estado local de la ejecución. HPKE base no autentica la identidad del exportador y el ensayo no demuestra autorización del usuario, attestation del proveedor ni persistencia global del estado antireplay.

Implementación del transporte

La implementación HPKE reproduce el vector oficial de RFC 9180 [3]. El Working Draft CXP [2] no define campos para la encapsulación HPKE ni para el challenge firmado que describe su narrativa, y no concreta completamente el ZIP/JWE. Usamos una extensión experimental autenticada como AAD y no reclamamos interoperabilidad CXP normativa completa.

Una segunda implementación, cryptography 50.0.1 [5], intercambió ciphertexts HPKE en ambas direcciones con PasskeyTransit. Un consumidor Node.js independiente validó el envelope CXF, PKCS#8, SPKI, credential ID y largeBlob DEFLATE. El ensayo híbrido ML-KEM-768+X25519 fue exitoso, pero permanece fuera del perfil CXP y de los estimandos.

Implementación CXF externa

La librería Rust credential-exchange-format 0.4.0 de Bitwarden, fijada al commit 0ee5516e4c0481ab6b0a68f8541fc39c3c3379b1, parseó y serializó un documento CXF con passkey y extensiones. El documento normalizado conservó exactamente su hash SHA-256. Esta prueba establece interoperabilidad de formato con una implementación abierta independiente; no ejecuta el flujo de un producto ni autoriza afirmaciones sobre Bitwarden como proveedor.

Resultados

Campaña C1 con navegador

De 6,144 intentos, 5,120 fueron importados y 1,024 rechazados por el control strict. Identidad, correspondencia de clave pública, aserción WebAuthn y UV pasaron en los intentos importados. Las dos repeticiones produjeron resultados semánticos equivalentes.

Clase semántica	N	Proporción
PASS	2,304	37.50%
Degradación visible	960	15.62%
Degradación silenciosa	1,344	21.88%
No evaluable	512	8.33%
Rechazo previo a ceremonia	1,024	16.67%

La tasa de degradación silenciosa fue 26.25% (1,344/5,120; intervalo descriptivo de sensibilidad 23.99%-28.42%). largeBlob fue observable en 1,216 casos aplicables: 704 pasaron y 512 fallaron según la ruta de control. La pérdida en un intermediario persistió al volver a un destino capaz, produciendo discordancias en comparaciones pareadas con el mismo destino.

Robustez C2

C2 ejecutó 80 casos: diez familias sobre ocho estratos. Se rechazaron 64 casos por validación estructural, política de duplicados o preservación estricta. Las familias se informan por separado; la clase normativa se deriva del requisito aplicable y del resultado observado. No se calcula un porcentaje agrupado.

Fallos y recuperación C3

C3 ejecutó 960 secuencias y 1920 eventos. En 832 secuencias (86.67%) hubo rollback completo y el retry convergió a una copia. Las 128 fallas restantes fueron el control positivo deliberado: legacy conservó un provisional en los dos puntos tardíos y el retry creó un duplicado, haciendo fallar atomicidad e idempotencia.

Discusión

El experimento demuestra una capacidad del método: la autenticación funcionó en los 5,120 intentos importados, incluidos aquellos en los que los controles descartaron otras propiedades. Los 1,024 rechazos strict detuvieron la importación antes de la ceremonia. Por tanto, un test de login aislado no es un oráculo suficiente para migración semántica.

También observamos dependencia de ruta: un destino final sin pérdidas propias no puede reconstruir material descartado por un intermediario. La declaración pre-commit cambia además la clasificación de una misma pérdida de silenciosa a visible, aun cuando el estado final de la credencial sea idéntico.

Limitaciones

- Los cuatro perfiles son controles sintéticos, no proveedores comerciales.
- CDP no permite inyectar HMAC/PRF ni credBlob; los casos positivos son NOT_EVALUABLE, nunca PASS.
- No ejecutamos Secure Payment Confirmation.
- El binding adicional de CXP es experimental y no resuelve autenticación de identidad del exportador en modo HPKE base.
- El ensayo PQC prueba disponibilidad criptográfica, no un perfil CXP-PQC.
- No se permite inferir vulnerabilidades, prevalencia de fallos ni superioridad de productos a partir de estos controles.

Reproducibilidad

Los artefactos raw son JSONL inmutables; los derivados y manifiestos incluyen hashes SHA-256 del protocolo, resultados, navegador y commit. La campaña C1 se ejecutó con Chromium 153.0.4234.48 y Playwright 1.62.0 desde un árbol Git limpio. El repositorio incluye comandos de una sola operación para tests, C1, C2, C3, interoperabilidad y regeneración de este análisis.

Conclusión

PasskeyTransit distingue compatibilidad sintáctica, autenticación básica, preservación funcional y recuperación operacional. En los controles diseñados, una aserción válida no basta para afirmar que una passkey migrada conserva todas sus garantías. El paso necesario para generalizar es ejecutar adaptadores identificados e independientes, manteniendo los mismos oráculos y límites de afirmación.

Referencias

1. FIDO Alliance, Credential Exchange Format v1.0 Proposed Standard Errata, 2026, <https://fidoalliance.org/specs/cx/cxf-v1.0-ps-errata-20260309.html>.

2. FIDO Alliance, Credential Exchange Protocol v1.0 Working Draft, 2024-10-03,
<https://fidoalliance.org/specs/cx/cxp-v1.0-wd-20241003.html>.
3. Barnes et al., Hybrid Public Key Encryption, RFC 9180, 2022, <https://www.rfc-editor.org/rfc/rfc9180>.
4. W3C, Web Authentication Level 3 Candidate Recommendation, 2026-05-26,
<https://www.w3.org/TR/2026/CR-webauthn-3-20260526/>.
5. PyCA, cryptography HPKE API documentation, <https://cryptography.io/en/latest/hazmat/primitives/hpke/>.
6. Bitwarden, credential-exchange v0.4.0, <https://github.com/bitwarden/credential-exchange/tree/v0.4.0>.
7. Jannett et al., The State of Passkeys, USENIX Security 2026,
<https://www.usenix.org/conference/usenixsecurity26/presentation/jannett>.